

TUTORIEL COMPLET BOT DISCORD

Construire un Bot a partir de zero avec Python

TABLE DES MATIERES

1. Introduction et Prerequis
2. Configuration Discord Developer
3. Configuration du Projet
4. Comprendre les Cogs et Commandes
5. Commandes Generales Expliquees
6. Commandes de Moderation Expliquees
7. Commandes de Gestion de Salons Expliquees
8. Sujets Avances et Bonnes Pratiques

1. INTRODUCTION ET PREREQUIS

Ce tutoriel vous guidera dans la creation d'un bot Discord complet en utilisant Python et la bibliotheque discord.py. Nous allons construire un bot avec plusieurs fonctionnalites incluant des commandes generales, des outils de moderation, et la gestion de salons.

Ce que vous apprendrez

- Comment configurer une application Discord
- Comprendre la structure de la bibliotheque discord.py
- Creer et organiser les commandes du bot avec les Cogs
- Implementer des fonctionnalites de moderation
- Gerer les permissions et la securite du bot

Prerequis

- Python 3.8 ou plus recent
- Un compte Discord
- Un serveur Discord pour les tests
- Connaissances basiques en Python
- VS Code ou tout autre editeur de texte

2. CONFIGURATION DISCORD DEVELOPER

Avant d'ecrire du code, vous devez creer une application Discord et obtenir un token de bot.

Etape 1: Creer votre Application

1. Visitez <https://discord.com/developers/applications>
2. Cliquez sur le bouton 'New Application'
3. Entrez le nom de votre bot et cliquez sur 'Create'
4. Vous avez maintenant un ID d'application dans la section General

Etape 2: Creer l'Utilisateur Bot

1. Allez dans l'onglet 'Bot'
2. Cliquez sur 'Add Bot'
3. Votre utilisateur bot est maintenant cree
4. Gardez les parametres par default pour l'instant

Etape 3: Obtenir Votre Token

1. Dans l'onglet 'Bot', cliquez sur 'Reset Token' ou afficher le token existant
2. Copiez le token (il ne sera plus jamais affiche!)
3. Stockez-le en toute securite - ne le committez jamais sur GitHub
4. Utilisez toujours un fichier .env pour stocker les donnees sensibles

3. CONFIGURATION DU PROJET

Configuration de la Structure du Projet

Creez la structure de dossiers suivante:

```
mon_bot_discord/  
main.py  
general.py  
moderation.py  
channels.py  
help_cog.py  
.env  
requirements.txt
```

Installation des Dependances

Creez un fichier requirements.txt avec:

```
discord.py==2.3.2  
python-dotenv==1.0.0
```

Puis installez: `pip install -r requirements.txt`

Configuration de votre Token

Creez un fichier .env a la racine de votre projet:

```
DISCORD_TOKEN=votre_token_ici
```

4. COMPRENDRE LES COGS ET COMMANDES

Qu'est-ce que les Cogs?

Les Cogs sont un moyen d'organiser les commandes de votre bot en modules separates. Chaque cog est une classe qui herite de `commands.Cog`. Cela maintient votre code propre et facilite la gestion d'un grand nombre de commandes.

Structure de Base d'un Cog

```
class MonCog(commands.Cog):  
    def __init__(self, bot):  
        self.bot = bot  
  
    @commands.command()  
    async def macommande(self, ctx):  
        await ctx.send('Bonjour!')  
  
    async def setup(bot):  
        await bot.add_cog(MonCog(bot))
```

5. COMMANDES GENERALES EXPLIQUEES

Commande: !ping

Objectif: Teste si le bot repond

Comment ca fonctionne: Quand un utilisateur tape !ping, le bot repond immediatement avec 'Pong!'

Implementation du code:

```
@commands.command()
async def ping(self, ctx):
    await ctx.send('Pong!')
```

Commande: !hello

Objectif: Salue l'utilisateur qui a appele la commande

Comment ca fonctionne: Le bot utilise ctx.author.mention pour obtenir la mention de l'utilisateur et le salue

Implementation du code:

```
@commands.command()
async def hello(self, ctx):
    await ctx.send(f'Salut {ctx.author.mention}!')
```

Commande: !info

Objectif: Affiche les informations du serveur

Comment ca fonctionne: Utilise les embeds Discord pour afficher les informations formatees du serveur

Informations affichees:

- Nom du serveur et nombre de membres
- Date de creation du serveur
- Proprietaire du serveur

6. COMMANDES DE MODERATION EXPLIQUEES

Commande: !kick @membre [raison]

Objectif: Expulse temporairement un membre du serveur

Permissions Requises: Expulser les membres

Comment ca fonctionne:

1. Verifie que l'utilisateur a les permissions
2. Empeche de vous kicker vous-meme ou des roles superieurs
3. Expulse le membre avec une raison optionnelle
4. Envoie un embed de confirmation au chat

Commande: !ban @membre [raison]

Objectif: Bannit definitivement un membre du serveur

Permissions Requises: Bannir les membres

Difference cle avec le kick: Le membre ne peut pas se rejoin sauf s'il est debanni

Commande: !unban nom#tag

Objectif: Annule un bannissement et permet au membre de se rejoin

Comment ca fonctionne:

1. Recupere la liste des bannissements du serveur
2. Recherche le membre par nom et discriminateur
3. Supprime le bannissement s'il est trouve

Commande: !mute @membre [raison]

Objectif: Empeche un membre d'envoyer des messages

Comment ca fonctionne:

1. Recherche un role nomme 'Muted'
2. Ajoute ce role au membre
3. Le role Muted doit avoir la permission d'envoyer des messages desactivee

Commande: !unmute @membre [raison]

Objectif: Enleve l'etouffement d'un membre

Comment ça fonctionne: Supprime le role Muted du membre

7. COMMANDES DE GESTION DE SALONS EXPLIQUEES

Commande: !lock [#salon]

Objectif: Empêche les membres d'envoyer des messages dans un salon

Permissions Requises: Gérer les salons

Comment ça fonctionne:

1. Crée une permission overwrite pour @everyone
2. Définit la permission send_messages à Faux
3. Les modérateurs peuvent toujours envoyer des messages
4. Utile lors des événements de modération ou des annonces

Commande: !unlock [#salon]

Objectif: Réactive l'envoi de messages dans un salon verrouillé

Comment ça fonctionne:

1. Supprime la permission overwrite
2. Restaure les permissions par défaut
3. Les membres peuvent envoyer des messages à nouveau

Commande: !lockall

Objectif: Verrouille TOUS les salons de texte du serveur à la fois

Cas d'usage: Blocage d'urgence lors de raids ou d'attaques de spam

Comment ça fonctionne:

1. Boucle à travers tous les salons de texte
2. Applique send_messages=False à chacun
3. Compte et signale le nombre de salons verrouillés

Commande: !unlockall

Objectif: Deverrouille TOUS les salons de texte à la fois

Cas d'usage: Restaurer le serveur après un blocage d'urgence

8. SUJETS AVANCES ET BONNES PRATIQUES

Verification des Permissions

Verifiez toujours les permissions de l'utilisateur avant d'executer des commandes sensibles:

```
@commands.has_permissions(kick_members=True)
```

Cela empeche les utilisateurs non autorises d'utiliser les commandes de moderation

Gestion des Erreurs

Enveloppez toujours les appels API Discord dans des blocs try-except:

```
try:
    await member.kick(reason=reason)
except discord.Forbidden:
    await ctx.send('Pas de permission')
```

Utiliser les Embeds pour une Meilleure Sortie

Les embeds offrent des messages formattes et colores qui ont l'air professionnel:

```
embed = discord.Embed(title='Titre', color=discord.Color.blue())
```

C'est beaucoup mieux que les reponses en texte brut

Bonnes Pratiques de Securite

- Ne codez jamais votre token en dur dans le code source
- Utilisez toujours des variables d'environnement (fichiers .env)
- Ajoutez .env et .gitignore au controle de version
- Regenez votre token s'il est jamais expose
- Limitez les permissions du bot a ce qui est reellement necessaire

CONCLUSION ET PROCHAINES ETAPES

Vous avez maintenant un bot Discord entierement fonctionnel avec plusieurs fonctionnalites. La structure modulaire utilisant les Cogs facilite l'ajout de plus de commandes a l'avenir.

Idees pour Elargir Votre Bot

- Creer un message de bienvenue pour les nouveaux membres
- Ajouter un systeme de journalisation pour les actions de moderation
- Creer des commandes amusantes (blague, citation, etc.)
- Implementer l'integration de bases de donnees
- Ajouter la moderation automatisee (detection de spam)
- Creer des commandes d'attribution de roles

Ressources

- Documentation discord.py: <https://discordpy.readthedocs.io/>
- Portail Developpeur Discord: <https://discord.com/developers/>
- Documentation Python: <https://docs.python.org/3/>